

Point Cloud Aggregation Based on GNSS-INS Data and Moving Object Filtering

1. Introduction

This assignment focuses on building a static, globally consistent 3D representation of an urban environment using LiDAR, GNSS-INS ego-motion, and camera data. The goal is threefold:

1. Aggregate frame-by-frame LiDAR measurements into a global coordinate system.
2. Remove measurements corresponding to moving objects without relying on any manual annotations.
3. Enhance the resulting static point cloud by assigning color values from camera images.

All methods are implemented in Python and validated using the **nuScenes v1.0-mini dataset**, which provides the same sensor configuration, calibration, and ego-pose format as the full nuScenes dataset.

2. Dataset and Sensor Configuration

The nuScenes dataset contains synchronized multi-sensor data recorded in urban environments. In this project, the following sensors are used:

- **LiDAR:** `LIDAR_TOP`
- **Cameras:** six RGB cameras covering a 360° field of view
- **Ego-motion:** GNSS-INS fused pose estimates (`ego_pose` records)

All sensors are intrinsically and extrinsically calibrated, and timestamps are synchronized across modalities.

3. Coordinate Frames and Transformations

Three coordinate frames are used throughout the pipeline:

- **LiDAR frame:** raw point cloud coordinates from the LiDAR sensor
- **Ego vehicle frame:** vehicle-centric reference frame
- **Global frame:** world coordinate system provided by nuScenes

For each LiDAR point, the following transformation chain is applied:

$$\mathbf{p}_{global} = T_{ego \rightarrow global} \cdot T_{lidar \rightarrow ego} \cdot \mathbf{p}_{lidar}$$

Rigid transformations are represented as 4×4 homogeneous matrices constructed from rotation quaternions and translations.

4. Task 1 – Point Cloud Aggregation

4.1 Method

Each LiDAR sweep is processed independently and then accumulated into a global point cloud:

1. Load LiDAR points from disk.
2. Transform points from the LiDAR frame to the ego frame using calibrated sensor extrinsics.
3. Transform points from the ego frame to the global frame using ego-pose data.
4. Accumulate transformed points across all sweeps.

To support later temporal filtering, each point is stored together with:

- its originating sweep index,
- its timestamp.

4.2 Implementation

The aggregation is implemented in the function:

```
aggregate_sweeps_to_global(sweeps)
```

The output is an `AggregatedCloud` structure containing:

- `xyz_global`: global 3D coordinates,
- `sweep_ids`: sweep index per point,
- `timestamps_us`: timestamp per point.

A lightweight voxel downsampling function is provided for visualization and performance purposes.

4.3 Output

- Aggregated global point cloud (`.ply`)
- Downsampled preview for visualization and video generation

5. Task 2 – Moving Object Filtering (Static Scene Extraction)

5.1 Motivation

Moving objects (vehicles, pedestrians) do not occupy the same spatial location across multiple LiDAR sweeps. In contrast, static structures (roads, buildings, poles) are observed repeatedly in the same global regions.

This observation enables moving object removal **without any semantic labels or bounding boxes**.

5.2 Temporal Voxel Consistency Filtering

The following strategy is applied:

1. Discretize the global point cloud into voxels of fixed size.
2. For each voxel, count how many **unique LiDAR sweeps** contributed points to it.
3. Mark a voxel as static if it is observed in at least K different sweeps.
4. Keep only points belonging to static voxels.

This removes transient measurements caused by moving objects.

5.3 Implementation

The filtering is implemented in:

```
temporal_voxel_static_mask(  
    xyz,  
    sweep_ids,  
    voxel_size,  
    min_unique_sweeps  
)
```

An optional radius-based outlier removal is applied to further suppress isolated noise.

5.4 Output

- Static point cloud (`static_filtered.ply`)
- Cleaned static point cloud after outlier removal (`static_filtered_clean.ply`)

6. Task 3 – Point Cloud Colorization

6.1 Overview

To improve interpretability and realism, the static point cloud is enriched with RGB color values using camera images.

6.2 Projection and Color Assignment

For each camera frame:

1. Transform global points into the camera coordinate frame.
2. Project 3D points onto the image plane using camera intrinsics.
3. Discard points behind the camera or outside the image bounds.
4. Use a per-image **z-buffer** to resolve occlusions.
5. Assign RGB values to visible points by sampling image pixels.

Points that are not visible in any camera remain uncolored.

6.3 Implementation

Colorization is implemented in:

```
colorize_points_from_cameras(  
    xyz_global,  
    camera_frames,  
    nusc  
)
```

Camera metadata and calibration are loaded using:

```
load_scene_camera_frames(...)
```

6.4 Output

- Colorized static point cloud (`static_colored.ply`)

7. Visualization and Video Generation

To visually demonstrate the results of each task, videos are generated by rendering point clouds using Open3D.

Due to platform limitations on macOS, a **visible OpenGL window** is used for rendering. The camera is smoothly rotated around the scene, and each frame is saved as a PNG image.

The image sequences are converted into MP4 videos using FFmpeg.

Generated videos:

- Task 1: Aggregated point cloud
- Task 2: Static point cloud after moving object removal
- Task 3: Colorized static point cloud

8. Notes on Dataset Usage

All experiments were conducted on the **nuScenes mini split**. This split contains the same sensor setup, calibration parameters, and ego-motion format as the full dataset, differing only in scale.

Therefore, the implemented pipeline is directly applicable to the full nuScenes dataset without modification.

9. Conclusion

This project demonstrates a complete LiDAR-based mapping pipeline using real autonomous driving data. By combining ego-motion, temporal consistency, and multi-view color projection, a clean and visually enriched static 3D representation of an urban environment is obtained, without relying on any manual annotations.